Reinventing Primality Testing through Randomized Algorithms

CS648 Project Presentation

Aaryan Maheshwari (230022) Aruz Awasthi (230209)
Md Arsal Asif (230643) Suyash Kapoor (231066)

Indian Institute of Technology Kanpur

April 5, 2026

Guided by Prof. Surender Baswana

Outline

1. Introduction & Motivation
2. Deterministic Algorithms
3. Fermat Primality Test
4. Miller–Rabin Test
5. Experimental Results
6. RSA Application & Error Contextualization
7. Comparison & Conclusion

Why Primality Testing?

The Core Problem

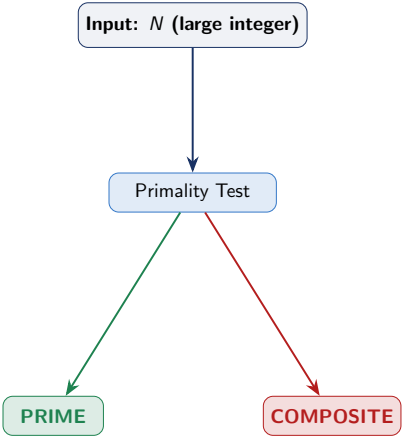
Given a number N, determine whether it is **prime** or **composite**.

Why does it matter?

- Foundation of **RSA** and **Diffie–Hellman** cryptography
- Secure communication needs large primes (1024–4096 bits)
- Must test *thousands* of candidates to find each prime

The Challenge

Deterministic methods are provably correct but *too slow* for large inputs.



Deterministic Algorithms — Overview

Trial Division

Check all $d \in [2, \lfloor \sqrt{N} \rfloor]$

$$T(N) = O\left(2^{n/2}\right)$$

Exponential in bits

Deterministic Algorithms — Overview

Trial Division

Check all $d \in [2, \lfloor \sqrt{N} \rfloor]$

$$T(N) = O\left(2^{n/2}\right)$$

Exponential in bits

Sieve of Eratosthenes

Mark multiples of each prime

$$T(N) = O(N \log \log N)$$

Bulk generation only

Deterministic Algorithms — Overview

Trial Division

Check all $d \in [2, \lfloor\sqrt{N}\rfloor]$

$$T(N) = O\left(2^{n/2}\right)$$

Exponential in bits

Sieve of Eratosthenes

Mark multiples of each prime

$$T(N) = O(N \log\log N)$$

Bulk generation only

AKS (IIT Kanpur, 2002)

First deterministic poly-time test

$$T(N) = O\left(\log^6 N\right)$$

Theoretically optimal, practically slow

Deterministic Algorithms — Overview

Trial Division

Check all $d \in [2, \lfloor \sqrt{N} \rfloor]$

$$T(N) = O\left(2^{n/2}\right)$$

Exponential in bits

Sieve of Eratosthenes

Mark multiples of each prime

$$T(N) = O(N \log \log N)$$

Bulk generation only

AKS (IIT Kanpur, 2002)

First deterministic poly-time test

$$T(N) = O\left(\log^6 N\right)$$

Theoretically optimal, practically slow

The Practical Gap

AKS hits a **1-second cutoff beyond 4-digit numbers** in experiments. The hidden constant in $O(\log^6 N)$ is *astronomically large*.

Deterministic Algorithms — Overview

Trial Division

Check all $d \in [2, \lfloor \sqrt{N} \rfloor]$

$$T(N) = O\left(2^{n/2}\right)$$

Exponential in bits

Sieve of Eratosthenes

Mark multiples of each prime

$$T(N) = O(N \log \log N)$$

Bulk generation only

AKS (IIT Kanpur, 2002)

First deterministic poly-time test

$$T(N) = O\left(\log^6 N\right)$$

Theoretically optimal, practically slow

The Practical Gap

AKS hits a **1-second cutoff beyond 4-digit numbers** in experiments. The hidden constant in $O(\log^6 N)$ is *astronomically large*.

$\Rightarrow$ **Randomized algorithms** dominate in practice.

Fermat's Little Theorem — The Foundation

Theorem (Fermat's Little Theorem)

For any prime p and integer a with $\gcd(a, p) = 1$:

$$a^{p-1} \equiv 1 \pmod{p}$$

Fermat's Little Theorem — The Foundation

Theorem (Fermat's Little Theorem)

For any prime p and integer a with $\gcd(a, p) = 1$:

$$a^{p-1} \equiv 1 \pmod{p}$$

Key proof idea: The set $S = \{a, 2a, \ldots, (p-1)a\}$ is a permutation of $\{1, 2, \ldots, p-1\}$ modulo p. Taking the product of both sides:

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p} \quad \Longrightarrow \quad a^{p-1} \equiv 1 \pmod{p}$$

Fermat's Little Theorem — The Foundation

Theorem (Fermat's Little Theorem)

For any prime p and integer a with $\gcd(a, p) = 1$:

$$a^{p-1} \equiv 1 \pmod{p}$$

Key proof idea: The set $S = \{a, 2a, \ldots, (p-1)a\}$ is a permutation of $\{1, 2, \ldots, p-1\}$ modulo p. Taking the product of both sides:

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p} \quad \Longrightarrow \quad a^{p-1} \equiv 1 \pmod{p}$$

Implication for Testing

Define $A_{\text{fermat}} = \{a \in \mathbb{Z} \mid 1 \leq a < n,\ a^{n-1} \equiv 1 \pmod{n}\}$.

Fermat's Little Theorem — The Foundation

Theorem (Fermat's Little Theorem)

For any prime p and integer a with $\gcd(a, p) = 1$*:*

$$a^{p-1} \equiv 1 \pmod{p}$$

Key proof idea: The set $S = \{a, 2a, \ldots, (p-1)a\}$ is a permutation of $\{1, 2, \ldots, p-1\}$ modulo p. Taking the product of both sides:

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p} \quad \Longrightarrow \quad a^{p-1} \equiv 1 \pmod{p}$$

Implication for Testing

Define $A_{\text{fermat}} = \{a \in \mathbb{Z} \mid 1 \leq a < n,\ a^{n-1} \equiv 1 \pmod{n}\}$.

If n is prime $\Rightarrow$ *all* valid a satisfy the condition.

Fermat's Little Theorem — The Foundation

Theorem (Fermat's Little Theorem)

For any prime p and integer a with $\gcd(a, p) = 1$*:*

$$a^{p-1} \equiv 1 \pmod{p}$$

Key proof idea: The set $S = \{a, 2a, \ldots, (p-1)a\}$ is a permutation of $\{1, 2, \ldots, p-1\}$ modulo p. Taking the product of both sides:

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p} \quad \Longrightarrow \quad a^{p-1} \equiv 1 \pmod{p}$$

Implication for Testing

Define $A_{\text{fermat}} = \{a \in \mathbb{Z} \mid 1 \leq a < n,\ a^{n-1} \equiv 1 \pmod{n}\}$.

If n is prime $\Rightarrow$ *all* valid a satisfy the condition.
If $a^{n-1} \not\equiv 1 \pmod{n} \Rightarrow n$ is **definitely composite**.

Partitioning the Search Space

Partition $A = \{1, \ldots, n-1\}$ into:

- $A' = \{a \mid \gcd(a, n) = 1\}$ *(units mod n)*
- $A'' = \{a \mid \gcd(a, n) > 1\}$ *(non-units)*

Partitioning the Search Space

Partition $A = \{1, \ldots, n-1\}$ into:

- $A' = \{a \mid \gcd(a, n) = 1\}$ *(units mod n)*
- $A'' = \{a \mid \gcd(a, n) > 1\}$ *(non-units)*

Theorem

$A_{fermat} \subseteq A'$

Proof: If $a^{n-1} \equiv 1 \pmod{n}$, then by Bézout's identity $\gcd(a, n) = 1$. $\square$

Partitioning the Search Space

Partition $A = \{1, \ldots, n-1\}$ into:

- $A' = \{a \mid \gcd(a, n) = 1\}$ *(units mod n)*
- $A'' = \{a \mid \gcd(a, n) > 1\}$ *(non-units)*

Theorem

$A_{fermat} \subseteq A'$

Proof: If $a^{n-1} \equiv 1 \pmod{n}$, then by Bézout's identity $\gcd(a, n) = 1$. $\square$

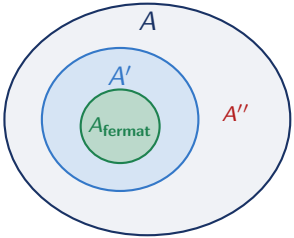
Consequence

Finding any a with $\gcd(a, n) > 1$ immediately proves n composite.

Partitioning the Search Space

Partition $A = \{1, \ldots, n-1\}$ into:

- $A' = \{a \mid \gcd(a, n) = 1\}$ *(units mod n)*
- $A'' = \{a \mid \gcd(a, n) > 1\}$ *(non-units)*



Theorem

$A_{fermat} \subseteq A'$

Proof: If $a^{n-1} \equiv 1 \pmod{n}$, then by Bézout's identity $\gcd(a, n) = 1$. $\square$

Consequence

Finding any a with $\gcd(a, n) > 1$ immediately proves n composite.

Half-density bound (non-Carmichael):
$|A_{\text{fermat}}| \leq \frac{|A'|}{2}$

Partitioning the Search Space

Partition $A = \{1, \ldots, n-1\}$ into:

- $A' = \{a \mid \gcd(a, n) = 1\}$ *(units mod n)*
- $A'' = \{a \mid \gcd(a, n) > 1\}$ *(non-units)*

Theorem

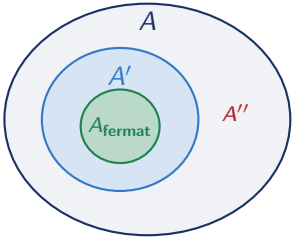
$A_{fermat} \subseteq A'$

Proof: If $a^{n-1} \equiv 1 \pmod{n}$, then by Bézout's identity $\gcd(a, n) = 1$. □

Consequence

Finding any a with $\gcd(a, n) > 1$ immediately proves n composite.

Half-density bound (non-Carmichael):
$|A_{\text{fermat}}| \leq \frac{|A'|}{2}$



Proof of half-density:

Let $x \in A' \setminus A_{\text{fermat}}$. The set xL (where $L = A_{\text{fermat}}$) is disjoint from L and has the same size. Since both are subsets of A':

$$|L| + |xL| \leq |A'| \implies |A_{\text{fermat}}| \leq \frac{|A'|}{2}$$

Carmichael Numbers — The Blind Spot

Problem: Carmichael Numbers

A composite n where $A' \subseteq A_{\text{fermat}}$:
every a with $\gcd(a, n) = 1$ satisfies the Fermat condition!

Carmichael Numbers — The Blind Spot

Problem: Carmichael Numbers

A composite n where $A' \subseteq A_{\text{fermat}}$:
every a with $\gcd(a, n) = 1$ satisfies the Fermat condition!

Examples: 561, 1105, 1729, 2465, ...

Carmichael Numbers — The Blind Spot

Problem: Carmichael Numbers

A composite n where $A' \subseteq A_{\text{fermat}}$:
every a with $\gcd(a, n) = 1$ satisfies the Fermat condition!

Examples: 561, 1105, 1729, 2465, ...

Theorem

For Carmichael n: $A' \setminus A_{\text{fermat}} = \emptyset$

Only elements in A'' (i.e., $\gcd(a, n) > 1$) can expose compositeness. For large n, $|A''|$ is tiny compared to n.

Carmichael Numbers — The Blind Spot

Problem: Carmichael Numbers

A composite n where $A' \subseteq A_{\text{fermat}}$:
every a with $\gcd(a, n) = 1$ satisfies the Fermat condition!

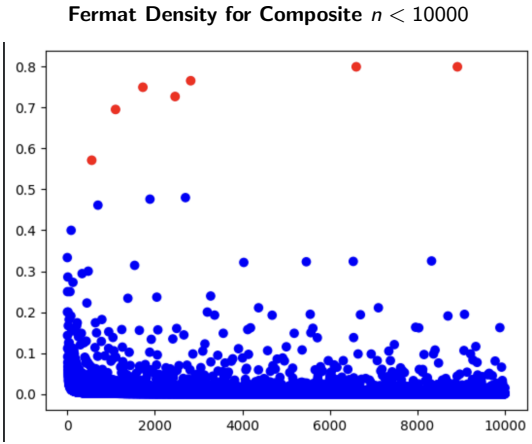
Examples: 561, 1105, 1729, 2465, ...

Theorem

For Carmichael n: $A' \setminus A_{\text{fermat}} = \emptyset$

Only elements in A'' (i.e., $\gcd(a, n) > 1$) can expose compositeness. For large n, $|A''|$ is tiny compared to n.

The Fermat test is fundamentally broken for Carmichael numbers.



Fermat Test — Algorithm & Error Analysis

Algorithm:
`Fermat_Primality_Test`(n, k)

```
if n is even then return COMPOSITE
end if
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    g ← gcd(a, n)
    if g > 1 then return COMPOSITE
    end if
    if a^(n−1) mod n ≠ 1 then return COMPOSITE
    end if
end for
return PRIME
```

Fermat Test — Algorithm & Error Analysis

Algorithm: Fermat_Primality_Test(n, k)

```
if n is even then return COMPOSITE
end if
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    g ← gcd(a, n)
    if g > 1 then return COMPOSITE
    end if
    if a^(n−1) mod n ≠ 1 then return COMPOSITE
    end if
end for
return PRIME
```

Error Analysis

Non-Carmichael composite:

$$P(\text{error}) \leq \left(\frac{1}{2}\right)^k$$

At $k = 40$: error $< 10^{-12}$

Fermat Test — Algorithm & Error Analysis

Algorithm: Fermat_Primality_Test(n, k)

```
if n is even then return COMPOSITE
end if
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    g ← gcd(a, n)
    if g > 1 then return COMPOSITE
    end if
    if a^(n−1) mod n ≠ 1 then return COMPOSITE
    end if
end for
return PRIME
```

Error Analysis

Non-Carmichael composite:

$$P(\text{error}) \leq \left(\frac{1}{2}\right)^k$$

At $k = 40$: error $< 10^{-12}$

Carmichael numbers:

$$P(\text{error}) \approx \left(\frac{|A'| - 2}{n - 3}\right)^k \approx 1$$

Test fails completely!

Fermat Test — Algorithm & Error Analysis

Algorithm: Fermat_Primality_Test(n, k)

```
if n is even then return COMPOSITE
end if
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    g ← gcd(a, n)
    if g > 1 then return COMPOSITE
    end if
    if a^(n−1) mod n ≠ 1 then return COMPOSITE
    end if
end for
return PRIME
```

Error Analysis

Non-Carmichael composite:

$$P(\text{error}) \leq \left(\frac{1}{2}\right)^k$$

At $k = 40$: error $< 10^{-12}$

Carmichael numbers:

$$P(\text{error}) \approx \left(\frac{|A'| - 2}{n - 3}\right)^k \approx 1$$

Test fails completely!

Time Complexity

$$O(k \cdot \log^3 n)$$

Modular exponentiation dominates.

The Key Insight — Square Roots of Unity

Theorem (Nontrivial Square Roots)

If n is an odd prime and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.

The Key Insight — Square Roots of Unity

Theorem (Nontrivial Square Roots)

If n is an odd prime and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.

Proof sketch: $x^2 \equiv 1 \Rightarrow n \mid (x-1)(x+1)$. Since n is prime, $n \mid (x-1)$ or $n \mid (x+1)$. $\square$

The Key Insight — Square Roots of Unity

Theorem (Nontrivial Square Roots)

If n is an odd prime and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.

Proof sketch: $x^2 \equiv 1 \Rightarrow n \mid (x-1)(x+1)$. Since n is prime, $n \mid (x-1)$ or $n \mid (x+1)$. □

Miller–Rabin's Improvement

Write $n - 1 = 2^s \cdot d$ with d odd. Inspect the squaring sequence:

$$a^d \to a^{2d} \to a^{4d} \to \cdots \to a^{2^s d} \equiv a^{n-1} \pmod{n}$$

- The sequence must end in 1 (by Fermat's Little Theorem)

The Key Insight — Square Roots of Unity

Theorem (Nontrivial Square Roots)

If n is an odd prime and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.

Proof sketch: $x^2 \equiv 1 \Rightarrow n \mid (x-1)(x+1)$. Since n is prime, $n \mid (x-1)$ or $n \mid (x+1)$. □

Miller–Rabin's Improvement

Write $n - 1 = 2^s \cdot d$ with d odd. Inspect the squaring sequence:

$$a^d \to a^{2d} \to a^{4d} \to \cdots \to a^{2^s d} \equiv a^{n-1} \pmod{n}$$

- The sequence must end in 1 (by Fermat's Little Theorem)
- The value *immediately before* the first 1 must be $n-1$ (i.e., -1)

The Key Insight — Square Roots of Unity

Theorem (Nontrivial Square Roots)

If n is an odd prime and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.

Proof sketch: $x^2 \equiv 1 \Rightarrow n \mid (x-1)(x+1)$. Since n is prime, $n \mid (x-1)$ or $n \mid (x+1)$. □

Miller–Rabin's Improvement

Write $n - 1 = 2^s \cdot d$ with d odd. Inspect the squaring sequence:

$$a^d \to a^{2d} \to a^{4d} \to \cdots \to a^{2^s d} \equiv a^{n-1} \pmod{n}$$

- The sequence must end in 1 (by Fermat's Little Theorem)
- The value *immediately before* the first 1 must be $n-1$ (i.e., -1)
- A violation $\Rightarrow$ **non-trivial root of 1** $\Rightarrow$ **composite**

Miller–Rabin — Algorithm & Analysis

Algorithm: `Miller_Rabin`(n, k)

```
Factor: n − 1 = 2^s · d,   d odd
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return COMPOSITE
        end if
        x ← y
    end for
    if x ≠ 1 then return COMPOSITE
    end if
end for
return PROBABLY PRIME
```

Miller–Rabin — Algorithm & Analysis

Algorithm: `Miller_Rabin`(n, k)

```
Factor: n − 1 = 2^s · d,   d odd
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return COMPOSITE
        end if
        x ← y
    end for
    if x ≠ 1 then return COMPOSITE
    end if
end for
return PROBABLY PRIME
```

Error & Complexity

Per iteration error bound:

$$P(\text{error per round}) \leq 1/4$$

After k rounds:

$$P(\text{error}) \leq 4^{-k}$$

Time complexity: $O(k \log^3 n)$

Miller–Rabin — Algorithm & Analysis

Algorithm: `Miller_Rabin`(n, k)

```
Factor: n − 1 = 2^s · d,   d odd
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return COMPOSITE
        end if
        x ← y
    end for
    if x ≠ 1 then return COMPOSITE
    end if
end for
return PROBABLY PRIME
```

Error & Complexity

Per iteration error bound:

$$P(\text{error per round}) \leq 1/4$$

After k rounds:

$$P(\text{error}) \leq 4^{-k}$$

Time complexity: $O(k \log^3 n)$

Advantages over Fermat

- **Immune** to Carmichael numbers

Miller–Rabin — Algorithm & Analysis

Algorithm: `Miller_Rabin`(n, k)

```
Factor: n − 1 = 2^s · d,   d odd
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return COMPOSITE
        end if
        x ← y
    end for
    if x ≠ 1 then return COMPOSITE
    end if
end for
return PROBABLY PRIME
```

Error & Complexity

Per iteration error bound:

$$P(\text{error per round}) \leq 1/4$$

After k rounds:

$$P(\text{error}) \leq 4^{-k}$$

Time complexity: $O(k \log^3 n)$

Advantages over Fermat

- **Immune** to Carmichael numbers
- Stronger per-round bound ($\frac{1}{4}$ vs $\frac{1}{2}$)

Miller–Rabin — Algorithm & Analysis

Algorithm: `Miller_Rabin`(n, k)

```
Factor: n − 1 = 2^s · d,   d odd
for i = 1 to k do
    Pick random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return COMPOSITE
        end if
        x ← y
    end for
    if x ≠ 1 then return COMPOSITE
    end if
end for
return PROBABLY PRIME
```

Error & Complexity

Per iteration error bound:

$$P(\text{error per round}) \leq 1/4$$

After k rounds:

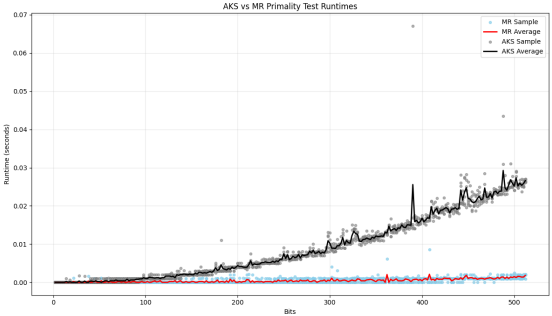
$$P(\text{error}) \leq 4^{-k}$$

Time complexity: $O(k \log^3 n)$

Advantages over Fermat

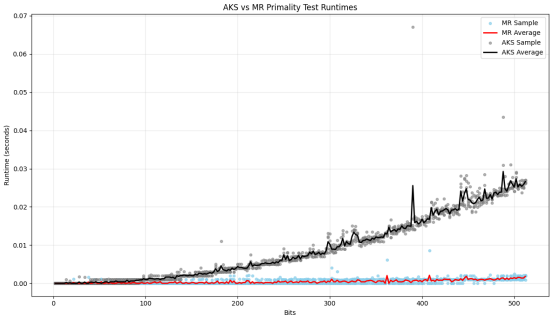
- **Immune** to Carmichael numbers
- Stronger per-round bound ($\frac{1}{4}$ vs $\frac{1}{2}$)
- Same asymptotic complexity

AKS vs Miller–Rabin: Runtime Comparison

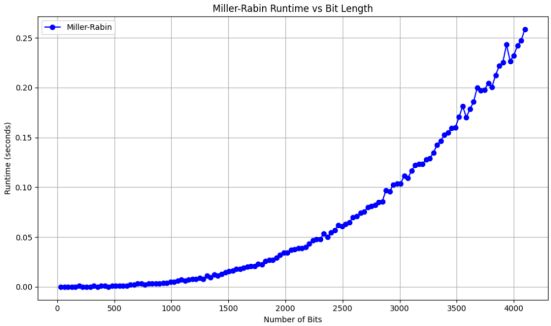


Miller-Rabin is almost flat relative to AKS

AKS vs Miller–Rabin: Runtime Comparison

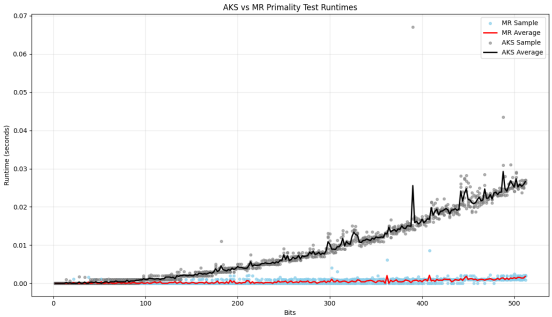


Miller-Rabin is almost flat relative to AKS

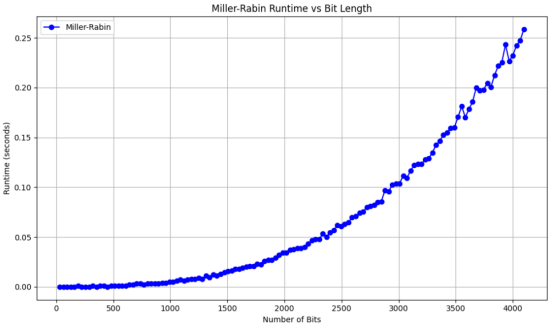


Handles 4096-bit inputs in only 0.25 s

AKS vs Miller–Rabin: Runtime Comparison



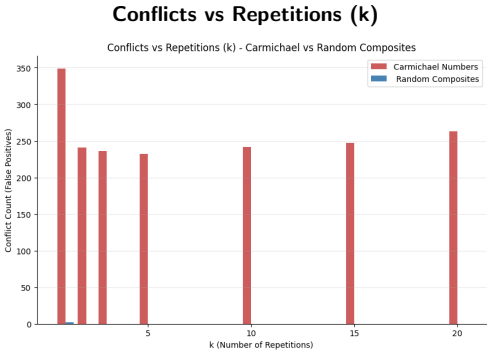
Miller-Rabin is almost flat relative to AKS



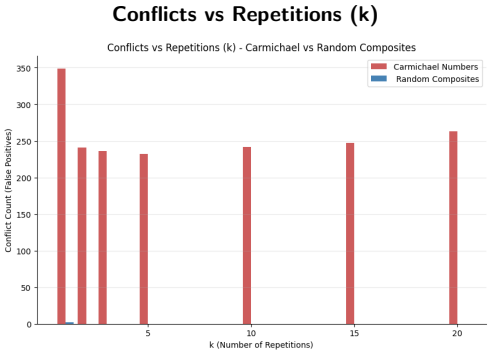
Handles 4096-bit inputs in only 0.25 s

Key takeaway: Miller-Rabin scales to cryptographic sizes; AKS does not.

False Positives: Carmichael vs Random Composites



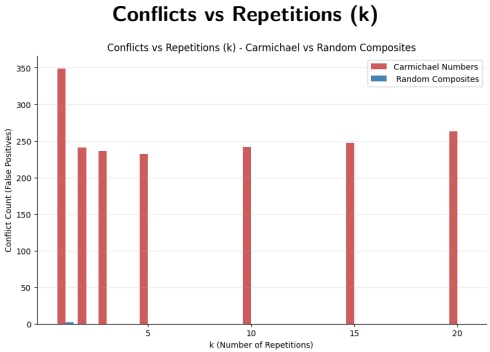
False Positives: Carmichael vs Random Composites



Observations:

- **Random composites** are almost never misidentified, even at low k

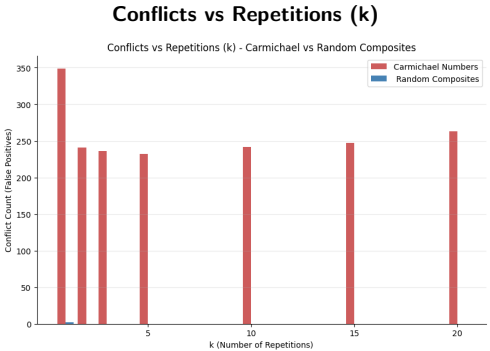
False Positives: Carmichael vs Random Composites



Observations:

- **Random composites** are almost never misidentified, even at low k
- **Carmichael numbers** cause more false positives at small k, but drop significantly as k grows

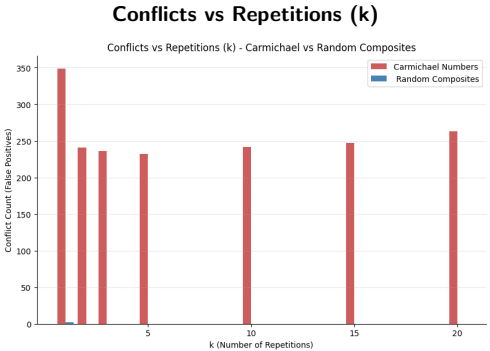
False Positives: Carmichael vs Random Composites



Observations:

- **Random composites** are almost never misidentified, even at low k
- **Carmichael numbers** cause more false positives at small k, but drop significantly as k grows
- Unlike Fermat, Miller-Rabin eventually catches Carmichael numbers too

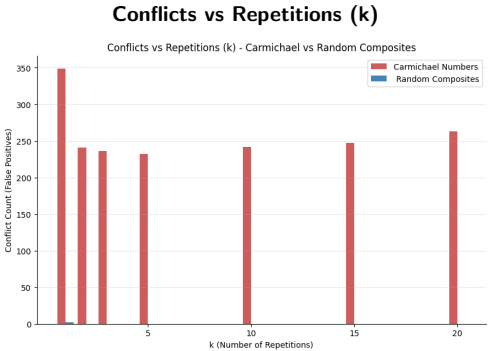
False Positives: Carmichael vs Random Composites



Observations:

- **Random composites** are almost never misidentified, even at low k
- **Carmichael numbers** cause more false positives at small k, but drop significantly as k grows
- Unlike Fermat, Miller-Rabin eventually catches Carmichael numbers too

False Positives: Carmichael vs Random Composites



Observations:

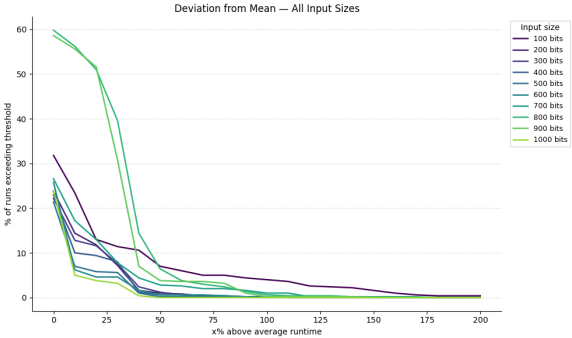
- **Random composites** are almost never misidentified, even at low k
- **Carmichael numbers** cause more false positives at small k, but drop significantly as k grows
- Unlike Fermat, Miller-Rabin eventually catches Carmichael numbers too

Why Miller-Rabin Wins

Nontrivial square root witnesses are abundant even for Carmichael numbers.

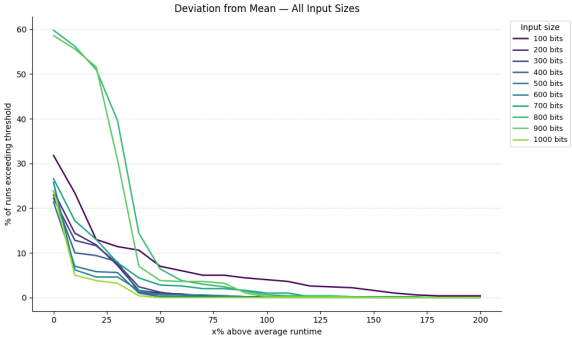
Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**.
Measured: % of runs exceeding mean by at least x%.



Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**. Measured: % of runs exceeding mean by at least x%.

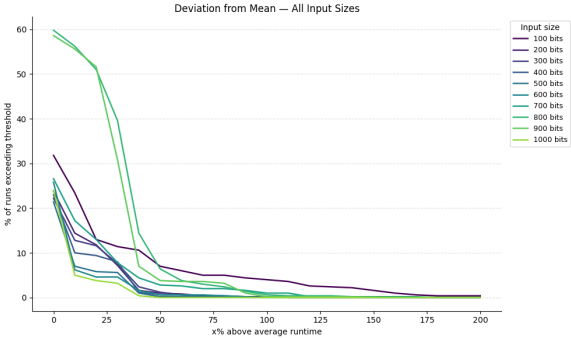


Key findings:

- **Small inputs (100–300 bits):** Higher relative spread due to timer resolution noise

Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**. Measured: % of runs exceeding mean by at least x%.

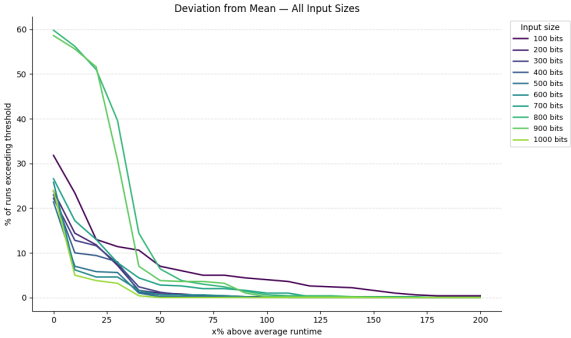


Key findings:

- **Small inputs (100–300 bits):** Higher relative spread due to timer resolution noise
- **Large inputs (600–1000 bits):** Sharp dropoff — actual computation dominates, behaviour is predictable

Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**. Measured: % of runs exceeding mean by at least x%.

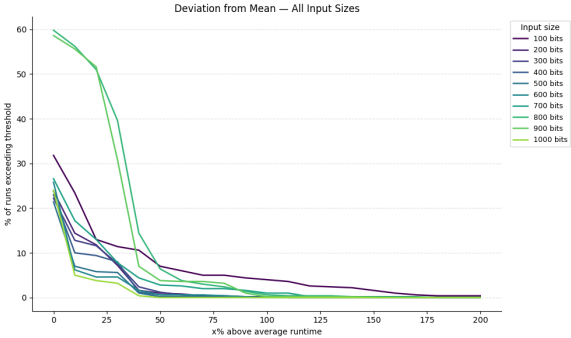


Key findings:

- **Small inputs (100–300 bits):** Higher relative spread due to timer resolution noise
- **Large inputs (600–1000 bits):** Sharp dropoff — actual computation dominates, behaviour is predictable
- Consistent with $O(k \log^3 n)$ complexity

Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**. Measured: % of runs exceeding mean by at least x%.

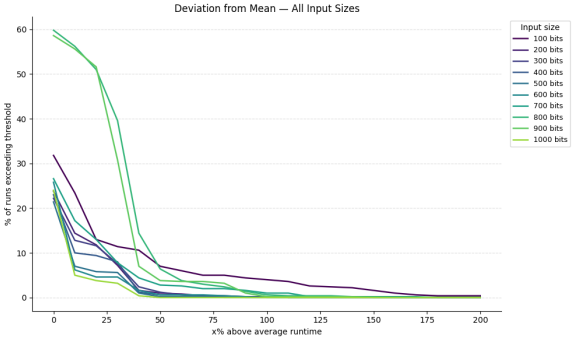


Key findings:

- **Small inputs (100–300 bits):** Higher relative spread due to timer resolution noise
- **Large inputs (600–1000 bits):** Sharp dropoff — actual computation dominates, behaviour is predictable
- Consistent with $O(k \log^3 n)$ complexity

Runtime Consistency — Deviation Analysis

Each input size benchmarked **2000 times**. Measured: % of runs exceeding mean by at least x%.



Key findings:

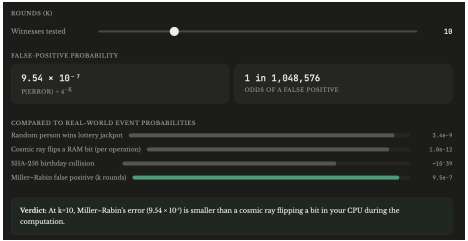
- **Small inputs (100–300 bits):** Higher relative spread due to timer resolution noise
- **Large inputs (600–1000 bits):** Sharp dropoff — actual computation dominates, behaviour is predictable
- Consistent with $O(k \log^3 n)$ complexity

Conclusion

Miller-Rabin is **highly predictable** at cryptographic scales.

Probabilistic Error in Context

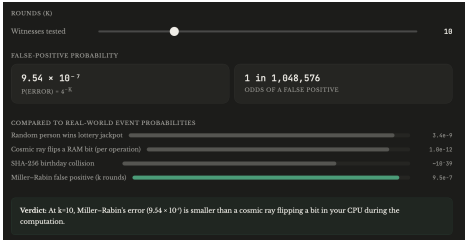
Error Probability Decay (4^{-k})



k (rounds)

Probabilistic Error in Context

Error Probability Decay (4^{-k})



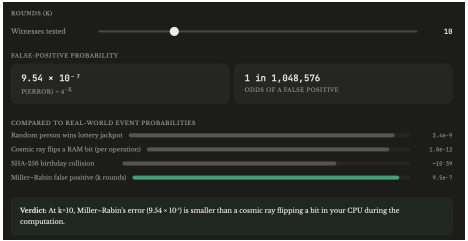
k (rounds)

Real-world comparison:

Event	Prob.
MR false positive ($k = 10$)	10^{-6}
MR false positive ($k = 20$)	10^{-12}
Cosmic ray RAM bit-flip	10^{-12}
Lottery jackpot win	10^{-9}
SHA-256 collision	10^{-39}

Probabilistic Error in Context

Error Probability Decay (4^{-k})



k (rounds)

Real-world comparison:

Event	Prob.
MR false positive ($k = 10$)	10^{-6}
MR false positive ($k = 20$)	10^{-12}
Cosmic ray RAM bit-flip	10^{-12}
Lottery jackpot win	10^{-9}
SHA-256 collision	10^{-39}

Practical Determinism

At $k = 20$, Miller-Rabin error is on par with *hardware failure*. At $k = 40$, it is considered **cryptographically safe**.

RSA Key Generation — Practical Impact

RSA-1024 key generation requires:

- Two 512-bit primes p and q
- By the Prime Number Theorem, ≈ 1 in 355 candidates is prime
- Thousands of primality tests per key generation

RSA Key Generation — Practical Impact

RSA-1024 key generation requires:

- Two 512-bit primes p and q
- By the Prime Number Theorem, ≈ 1 in 355 candidates is prime
- Thousands of primality tests per key generation

Simulation Results (496 candidates tested)

Algorithm	Time	Speedup
Miller-Rabin	130 ms	$1\times$
AKS	8.57 hours	$237{,}000\times$ slower

At 16 bits: Miller-Rabin is $50{,}923\times$ faster.

RSA Key Generation — Practical Impact

RSA-1024 key generation requires:

- Two 512-bit primes p and q
- By the Prime Number Theorem, ≈ 1 in 355 candidates is prime
- Thousands of primality tests per key generation

RSA-1024 Key Generation

↓

Miller-Rabin: 130 ms

↓

AKS: $\approx$ 8.6 hours

237,000× **speedup**
in favour of Miller-Rabin

Simulation Results (496 candidates tested)

Algorithm	Time	Speedup
Miller-Rabin	130 ms	1×
AKS	8.57 hours	237,000× slower

At 16 bits: Miller-Rabin is 50,923× faster.

Algorithm Comparison Summary

Property	**Trial Division**	**AKS**	**Miller-Rabin**
Complexity	$O(2^{n/2})$	$O(\log^6 N)$	$O(k \log^3 N)$
Deterministic?	✓	✓	× (prob.)
Handles Carmichael?	✓	✓	✓
Practical at 512 bits?	×	×	✓
Error at $k = 40$	N/A	N/A	$< 10^{-24}$
RSA-1024 generation	∞	∼8.6 hrs	∼130 ms

Algorithm Comparison Summary

Property	Trial Division	AKS	Miller-Rabin
Complexity	$O(2^{n/2})$	$O(\log^6 N)$	$O(k \log^3 N)$
Deterministic?	✓	✓	× (prob.)
Handles Carmichael?	✓	✓	✓
Practical at 512 bits?	×	×	✓
Error at $k = 40$	N/A	N/A	$< 10^{-24}$
RSA-1024 generation	∞	∼8.6 hrs	∼130 ms

AKS

Landmark theorem (IIT Kanpur, 2002). But hidden constant makes it impractical for real cryptographic use.

Algorithm Comparison Summary

Property	Trial Division	AKS	Miller-Rabin
Complexity	$O(2^{n/2})$	$O(\log^6 N)$	$O(k \log^3 N)$
Deterministic?	✓	✓	× (prob.)
Handles Carmichael?	✓	✓	✓
Practical at 512 bits?	×	×	✓
Error at $k = 40$	N/A	N/A	$< 10^{-24}$
RSA-1024 generation	∞	~8.6 hrs	~130 ms

AKS

Landmark theorem (IIT Kanpur, 2002). But hidden constant makes it impractical for real cryptographic use.

Miller-Rabin

Probabilistic yet *effectively deterministic* at $k \geq 20$. The de-facto standard in modern cryptography.

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$
4. At $k = 20$, error $\approx$ probability of a cosmic ray flipping a RAM bit

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$
4. At $k = 20$, error $\approx$ probability of a cosmic ray flipping a RAM bit
5. **AKS** is a landmark but 237,000$\times$ slower than Miller-Rabin for RSA

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$
4. At $k = 20$, error $\approx$ probability of a cosmic ray flipping a RAM bit
5. **AKS** is a landmark but 237,000$\times$ slower than Miller-Rabin for RSA
6. Randomized algorithms are the **backbone of modern cryptography**

Conclusion

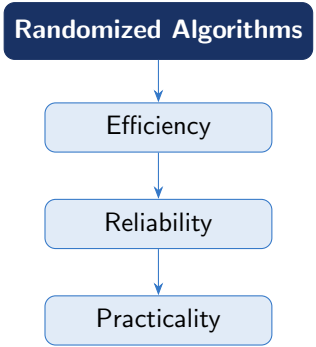
Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$
4. At $k = 20$, error $\approx$ probability of a cosmic ray flipping a RAM bit
5. **AKS** is a landmark but 237,000$\times$ slower than Miller-Rabin for RSA
6. Randomized algorithms are the **backbone of modern cryptography**

Conclusion

Key Takeaways

1. **Fermat's test** is elegant but broken for Carmichael numbers
2. **Miller-Rabin** overcomes this via nontrivial square root witnesses
3. Error decays **exponentially**: $\mathbb{P}(\text{error}) \leq 4^{-k}$
4. At $k = 20$, error $\approx$ probability of a cosmic ray flipping a RAM bit
5. **AKS** is a landmark but 237,000$\times$ slower than Miller-Rabin for RSA
6. Randomized algorithms are the **backbone of modern cryptography**



Thank You

Aaryan Maheshwari (230022) **Md Arsal Asif** (230643)
Aruz Awasthi (230209) **Suyash Kapoor** (231066)

Guided by Prof. Surender Baswana, IIT Kanpur